Pipeline and Concepts

Understand what you are building before you build it

1. The two-stage idea

Stage	Question it answers	Model	Input	Output
Stage 1: detector	Is there a plant, and where?	YOLOv8n (single class: plant)	Full camera frame (640 px)	Boxes + confidence
Stage 2: classifier	Healthy or diseased? Which disease?	MobileNetV3-Small	Close-up crop (224x224)	Class + confidence

Why two stages? A rover sees soil, people, pots and many other things. Stage 1 filters them out so Stage 2 only judges real plants. An earlier version of our detector, trained without negative images, called people 'plants'. That failure is a nice honest lesson to include in the paper.

2. Why 'universal' is a trap, and what we claim instead

Public datasets such as PlantVillage use single leaves on plain backgrounds. Models score very high on them but usually drop a lot on real field photos (sun, mud, other leaves). We therefore **claim only 'tomato-focused, multi-class'** and we **measure the drop** on field images. Reporting that gap makes the paper trustworthy.

3. Metrics you must understand and report

Metric	Plain meaning	Report for
Precision	Of everything the model flagged, how much was right? (few false alarms)	Detector, classifier
Recall	Of everything that was really there, how much did it find? (few misses)	Detector, classifier
mAP@0.5	Detector quality when a box counts as correct if it overlaps the true box by at least 50%	Detector
mAP@0.5:0.95	Stricter average over overlap thresholds 0.5 to 0.95	Detector
Accuracy	Share of images classified correctly	Classifier
Macro-F1	Average F1 over classes, so rare diseases count as much as common ones	Classifier
Confusion matrix	Table showing which classes get mixed up	Classifier (figure)
Latency (ms)	Time for one prediction; report median and 95th percentile	Both + camera
Mean +/- std	Run 3 seeds and report average and spread so results are not luck	Classifier

4. Three test sets (this is what makes the paper strong)

- **In-domain:** held-out PlantVillage tomato plants (easy, lab images).
- **Cross-domain:** PlantDoc tomato images (real field photos from the web). Expect a clear drop.
- **Deployment:** your own photos taken with the ESP32-CAM. This is the test that proves it works on our hardware.

> **WATCH OUT:** Never test on images the model saw during training. Never tune settings by looking at the test set; use the validation set for that.

5. Where does the model run?

For the bench tests, the ESP32-CAM sends a JPEG over WiFi to a laptop (or Colab), which runs the models. The ESP32-CAM itself cannot run these models. In the final rover, a server (cloud or laptop) will run them and send alerts to the dashboard. In the paper we say exactly where inference ran (e.g. 'laptop CPU, i5-xxxx').

6. Words to know

Term	Meaning
Fine-tuning	Start from a model already trained on ImageNet and adjust it on our tomato data (needs far less data)
Augmentation	Randomly flip, rotate, brighten images during training so the model sees more variety
Overfitting	Model memorises training images and fails on new ones; training accuracy high, test accuracy low
Data leakage	Test images that are near-copies of training images (e.g. same leaf photographed twice). Gives fake high accuracy
Hard negatives	Images without plants (people, soil, pots) used to teach the detector what NOT to flag
Domain gap	Difference between training images (lab) and real use images (field)